

```
import re
import sys
import joblib
import pandas as pd
```

```
from sklearn.model_selection import train_test_split
from sklearn.pipeline import FeatureUnion, Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score
```

```
DATASET = "data/train.csv"
MODEL_FILE = "models/ai_detector.joblib"
```

```
def clean_text(text):
    text = str(text)
    text = re.sub(r"\s+", " ", text)
    return text.strip()
```

```
def train():
    data = pd.read_csv(DATASET)
```

```
data = data.dropna(subset=["text", "label"])
data["text"] = data["text"].apply(clean_text)
data["label"] = data["label"].str.lower().str.strip()
```

```
data = data[data["label"].isin(["human", "ai"])]
```

```
X = data["text"]
y = data["label"]
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    stratify=y,
    random_state=42
)
```

```
word_features = TfidfVectorizer(
    analyzer="word",
    ngram_range=(1, 3),
    min_df=2,
    max_df=0.98,
    sublinear_tf=True,
    max_features=100000
)
```

```
character_features = TfidfVectorizer(  
    analyzer="char_wb",  
    ngram_range=(3, 6),  
    min_df=2,  
    sublinear_tf=True,  
    max_features=100000  
)
```

```
features = FeatureUnion([  
    ("word", word_features),  
    ("character", character_features)  
])
```

```
model = Pipeline([  
    ("features", features),  
    ("classifier", LogisticRegression(  
        max_iter=3000,  
        class_weight="balanced",  
        C=2.0  
    ))  
])
```

```
model.fit(X_train, y_train)
```

```
predictions = model.predict(X_test)
```

```
probabilities = model.predict_proba(X_test)
```

```
print("\nCLASSIFICATION REPORT\n")  
print(classification_report(y_test, predictions))
```

```
print("\nCONFUSION MATRIX\n")  
print(confusion_matrix(y_test, predictions))
```

```
ai_index = list(model.classes_).index("ai")  
ai_probability = probabilities[:, ai_index]
```

```
y_binary = (y_test == "ai").astype(int)
```

```
print(  
    "\nROC-AUC:",  
    round(roc_auc_score(y_binary, ai_probability), 4)  
)
```

```
joblib.dump(model, MODEL_FILE)
```

```
print(f"\nModel saved to: {MODEL_FILE}")
```

```
def detect(text):  
    model = joblib.load(MODEL_FILE)
```

```
text = clean_text(text)

probabilities = model.predict_proba([text])[0]
classes = model.classes_

scores = dict(zip(classes, probabilities))

ai_score = scores.get("ai", 0)
human_score = scores.get("human", 0)

if ai_score >= 0.75:
    result = "AI-LIKE"
elif ai_score <= 0.25:
    result = "HUMAN-LIKE"
else:
    result = "UNCERTAIN"

print("\n=====")
print("    AI TEXT DETECTOR")
print("=====")

print(f"\nResult    : {result}")
print(f"AI score    : {ai_score:.2%}")
print(f"Human score  : {human_score:.2%}")
print(f"Word count   : {len(text.split())}")
```

```
print("\n=====\\n")
```

```
def interactive():
```

```
    print("AI Text Detector")
```

```
    print("Type 'exit' to quit.\\n")
```

```
    while True:
```

```
        text = input("Enter text: ")
```

```
        if text.lower().strip() == "exit":
```

```
            break
```

```
        if not text.strip():
```

```
            print("Please enter some text.\\n")
```

```
            continue
```

```
        detect(text)
```

```
if __name__ == "__main__":
```

```
    if len(sys.argv) > 1:
```

```
        if sys.argv[1] == "train":
```

```
            train()
```

```
elif sys.argv[1] == "detect":  
    text = " ".join(sys.argv[2:])  
    detect(text)
```

```
else:  
    print("Usage:")  
    print("python ai_detector.py train")  
    print("python ai_detector.py detect \"your text here\"")
```

```
else:  
    interactive()
```